

RAPPORT DE CONCEPTION

Mini Système de Gestion de fichiers (SGF)
((Émulateur de système de fichier inspiré de UNIX/Linux))

Réalisé par :

Marc ELYSÉE

Yassine JOUDI

Ahmed ISHAGH

Ayman MSIMAR

Encadré par :

Mme Dhekra Abouda

Projet IATIC3 à effectuer du 01/04/2026 au 29/05/2026

Année universitaire : **2025/2026**

Table des matières

Partie I – Rapport d’analyse (mis à jour)	3
1 Analyse des besoins et objectifs	3
1.1 Besoins fonctionnels	3
1.2 Contraintes techniques	3
2 Architecture du système	4
3 Structures de données prévues	4
3.1 Superbloc	4
3.2 Structure Inode	4
3.3 Structure Bloc	4
3.4 Structure Répertoire	4
4 Liste des fonctions principales (cahier des charges initial)	4
4.1 Primitives système – gestion des fichiers	5
4.2 Primitives système – gestion des répertoires	5
4.3 Commandes Shell prévues	5
Partie II – Rapport de conception (mis à jour)	6
5 Organisation du fichier « disque »	6
5.1 Constantes du SGF	6
6 Structures de données détaillées	6
6.1 Le superbloc	6
6.2 La structure Inode	6
6.3 La structure Bloc et le format des répertoires	7
7 Algorithmes des fonctions principales	7
7.1 Algorithme mycreat(nom, mode)	7
7.2 Algorithme mywrite(inode_num, buffer, nombre)	7
7.3 Algorithme myunlink(nom) avec compteur de liens	8
7.4 Boucle du Shell	8
7.5 Sauvegarde et recharge	8
Partie III – Implémentation (nouvelle partie)	9
8 Organisation finale du code source	9
9 Détails de l’implémentation	9
9.1 Gestion du disque simulé	9
9.2 Primitives fichiers	9
9.3 Navigation et primitives répertoires	10

9.4	Mécanisme des liens	10
9.5	Interpréteur Shell	10
9.6	Métadonnées et commande <code>ls -l</code>	10
10	Démonstration : captures d'exécution	10
10.1	Compilation via le Makefile	10
10.2	Lancement et création de fichiers	11
10.3	Navigation dans l'arborescence	11
10.4	Copie, déplacement et liens	11
10.5	Informations sur le disque (<code>df</code>)	12
10.6	Persistance : preuve par l'exemple	12
11	Tests de validation	12
11.1	Catégories de tests couvertes	13
11.2	Résultat de l'exécution du script	13
11.3	Qualité du code	14
	Partie IV – Planning et répartition des tâches	15
12	Planning du projet	15
13	Répartition des tâches	15
14	Récapitulatif des évolutions	16
15	Difficultés rencontrées et solutions	17

Partie I – Rapport d’analyse (mis à jour)

Introduction

Cette première partie reprend le rapport d’analyse remis le 20 avril 2026. Elle définit les besoins de l’utilisateur, l’architecture envisagée, les structures de données prévues, ainsi que la répartition initiale des tâches.

1 Analyse des besoins et objectifs

L’objectif principal de ce projet est de concevoir un émulateur de système de fichiers inspiré de l’architecture UNIX/Linux, reposant sur la gestion d’inodes et de blocs de données. L’utilisateur doit pouvoir interagir avec ce système par l’intermédiaire d’un interpréteur de commandes (Shell) développé spécifiquement pour ce projet.

1.1 Besoins fonctionnels

- **Abstraction du stockage** : simuler un disque dur à l’aide d’un fichier binaire unique stocké sur la machine hôte.
- **Hiérarchie** : gérer une arborescence complète comprenant une racine (/), des répertoires et des sous-répertoires.
- **Manipulation de fichiers** : permettre la création, la lecture, l’écriture, la suppression, la copie, le déplacement et la création de liens vers des fichiers.
- **Manipulation de répertoires** : permettre la création (mkdir), la suppression (rmdir) et le parcours (cd, ls) des répertoires.
- **Interface utilisateur** : fournir un terminal (Mini-Bash) capable d’interpréter des commandes textuelles, incluant la redirection (echo "texte" > fichier).
- **Gestion des métadonnées** : associer à chaque fichier des droits d’accès (lecture / écriture / exécution) et une date de modification.
- **Persistance** : sauvegarder l’état du SGF à la fermeture du programme dans un fichier image, et le recharger au démarrage suivant.
- **Validation** : prévoir des tests vérifiant la cohérence du SGF en s’appuyant sur le mécanisme de sauvegarde / recharge.

1.2 Contraintes techniques

- Le projet est développé intégralement en langage C.
- Le code est modularisé : les primitives, les commandes shell, la sauvegarde/recharge et la structure du SGF sont placées dans des fichiers distincts.

- Chaque fonction est commentée afin de faciliter la lecture et la maintenance.
- La compilation est automatisée via un Makefile et le versionnage est assuré par Git/GitHub.

2 Architecture du système

Afin de garantir une modularité maximale et faciliter la maintenance du code, le projet est découpé en quatre couches logiques distinctes :

1. **Couche matérielle simulée (le fichier « disque »)** : un fichier de taille fixe, segmenté en deux zones, une réservée aux inodes et une dédiée aux blocs de données.
2. **Couche SGF (gestion interne)** : prend en charge l'allocation des blocs, la table des inodes et la structure des répertoires.
3. **Couche primitives (API)** : regroupe les fonctions de bas niveau (myopen, myread, mywrite, etc.) qui assurent le lien entre le Shell et la structure interne du SGF.
4. **Couche Shell (interface)** : boucle de type Read-Eval-Print Loop (REPL) qui interprète les saisies de l'utilisateur et les transforme en appels aux primitives système.

3 Structures de données prévues

L'implémentation s'appuie sur des structures définies en langage C, utilisées pour simuler le fonctionnement bas niveau du matériel.

3.1 Superbloc

Contient les métadonnées globales du système : taille totale du disque, nombre d'inodes libres, adresse du premier bloc de données.

3.2 Structure Inode

Chaque inode représente un fichier ou un répertoire et contient : le type (fichier texte, binaire, répertoire), la taille en octets, le tableau d'index des blocs, les droits d'accès (rwxrwxrwx), la date de modification, et le compteur de liens.

3.3 Structure Bloc

Tableau d'octets de taille fixe (1024 octets), unité élémentaire de stockage.

3.4 Structure Répertoire

Fichier spécial contenant la liste des couples (nom du fichier, numéro d'inode).

4 Liste des fonctions principales (cahier des charges initial)

4.1 Primitives système – gestion des fichiers

Primitive	Description
<code>mycreat(nom, mode)</code>	Crée un fichier et retourne le numéro de son inode.
<code>myopen(nom, mode)</code>	Ouvre un fichier existant (ou le crée si absent) et retourne son inode.
<code>myclose(inode)</code>	Ferme le fichier identifié par son inode.
<code>myread(inode, buffer, nombre)</code>	Lit nombre octets depuis le fichier et les place dans buffer.
<code>mywrite(inode, buffer, nombre)</code>	Écrit nombre octets depuis buffer dans le fichier, en allouant de nouveaux blocs si nécessaire.

4.2 Primitives système – gestion des répertoires

Primitive	Description
<code>mymkdir(nom)</code>	Crée un nouveau répertoire dans le répertoire courant.
<code>myrmdir(nom)</code>	Supprime un répertoire, à condition qu'il soit vide.
<code>mylink(nom1, nom2)</code>	Crée une nouvelle entrée de répertoire (nom2) pointant vers un fichier existant (nom1).
<code>myunlink(nom)</code>	Efface une entrée du répertoire; si plus aucune entrée ne référence le fichier, son inode et ses blocs sont libérés.

4.3 Commandes Shell prévues

Commandes fichiers : `cp`, `rm`, `mv`, `cat`, `ln`, `echo "texte" > file`. Commandes répertoires : `ls`, `mkdir`, `rmdir`, `cd`. Commande système : `df`.

Partie II – Rapport de conception (mis à jour)

Introduction

Cette deuxième partie reprend le rapport de conception remis le 2 mai 2026. Elle décrit en détail l'organisation du fichier disque, les structures internes et les algorithmes (en pseudo-code) des fonctions principales.

5 Organisation du fichier « disque »

Le SGF est entièrement contenu dans un unique fichier binaire de taille fixe nommé `disk.img`. Ce fichier joue le rôle d'un disque dur virtuel et est segmenté en trois zones contiguës : le superbloc en tête, suivi de la table des inodes, puis de la zone des blocs de données.

5.1 Constantes du SGF

- `NB_INODES` = 64
- `NB_BLOCS` = 256
- `TAILLE_BLOC` = 1024 octets
- `MAX_BLOCS_FIC` = 30 (taille du tableau d'index dans l'inode)
- `MAX_NOM` = 32 caractères

6 Structures de données détaillées

Toutes les structures sont de taille fixe afin de permettre un accès direct par offset dans le fichier image.

6.1 Le superbloc

```
Structure Superbloc :  
- nb_inodes_libres      : entier  
- nb_blocs_libres      : entier  
- inode_racine         : entier (numero d'inode de la racine /)  
- bitmap_inodes[64]    : tableau de booléens (1 = utilise, 0 = libre)  
- bitmap_blocs[256]    : tableau de booléens (1 = utilise, 0 = libre)
```

6.2 La structure Inode

```
Structure Inode :  
- type                  : entier (0 = libre, 1 = fichier, 2 =  
    repertoire)  
- taille                : entier (en octets)  
- droits                : entier (permissions au format rwxrwxrwx)  
- date_modif            : entier (timestamp Unix)  
- nb_liens              : entier (compteur pour link/unlink)
```

```
- blocs[30]          : tableau d'entiers (numeros des blocs ; -1 si vide)
```

6.3 La structure Bloc et le format des répertoires

Un bloc est un tableau de 1024 octets. Un répertoire est un fichier spécial dont les blocs contiennent une suite d'entrées (nom, inode). Chaque répertoire contient au minimum les deux entrées . (lui-même) et .. (son parent).

7 Algorithmes des fonctions principales

7.1 Algorithme mycreat(nom, mode)

```
Fonction mycreat(nom, mode) :
  Si la longueur de nom > MAX_NOM Alors Retourner -1 FinSi
  Si une entree de ce nom existe deja dans le repertoire courant
    Alors Retourner -1 (fichier deja existant) FinSi
  i <- chercher un inode libre dans bitmap_inodes
  Si i = -1 Alors Retourner -1 (plus d'inodes disponibles) FinSi
  Marquer bitmap_inodes[i] comme utilise
  inode[i].type      <- 1 (fichier)
  inode[i].taille    <- 0
  inode[i].droits     <- mode
  inode[i].date_modif<- date courante
  inode[i].nb_liens  <- 1
  Pour j de 0 a 29 : inode[i].blocs[j] <- -1 FinPour
  Ajouter (nom, i) dans le repertoire courant
  Decrementer nb_inodes_libres
  Retourner i
```

7.2 Algorithme mywrite(inode_num, buffer, nombre)

```
Fonction mywrite(inode_num, buffer, nombre) :
  octets_ecrits <- 0 ; reste <- nombre
  Pour j de 0 a 29 et tant que reste > 0 :
    Si inode[inode_num].blocs[j] = -1 Alors
      b <- chercher un bloc libre
      Si b = -1 Alors Sortir (disque plein) FinSi
      Marquer bitmap_blocs[b] comme utilise
      inode[inode_num].blocs[j] <- b
    FinSi
    a_ecrire <- min(reste, TAILLE_BLOC)
    Copier a_ecrire octets vers bloc[inode[inode_num].blocs[j]]
    octets_ecrits <- octets_ecrits + a_ecrire
    reste <- reste - a_ecrire
  FinPour
  inode[inode_num].taille <- max(taille actuelle, octets_ecrits)
  Retourner octets_ecrits
```


7.3 Algorithme myunlink(nom) avec compteur de liens

```
Fonction myunlink(nom) :  
  ino <- supprimer l'entree (nom) du repertoire courant  
  Si ino = -1 Alors Retourner -1 FinSi  
  Decrementer inode[ino].nb_liens  
  Si inode[ino].nb_liens <= 0 Alors  
    Libérer tous les blocs utilisés par inode[ino]  
    Marquer bitmap_inodes[ino] comme libre  
  FinSi  
  Retourner 0
```

7.4 Boucle du Shell

```
Fonction shell() :  
  Tant Que vrai Faire  
    Afficher '[mysh] $ '  
    ligne <- lire une ligne au clavier  
    args <- decouper la ligne en mots (espace = separateur)  
    Si args[0] = 'exit' Alors save_disk() ; Sortir FinSi  
    dispatcher(args)  
  FinTant
```

7.5 Sauvegarde et recharge

```
Fonction save_disk() :  
  f <- ouvrir 'disk.img' en ecriture binaire  
  Ecrire la structure Disque (superbloc + inodes + blocs) dans f  
  Fermer f  
  
Fonction load_disk() :  
  Si 'disk.img' n'existe pas Alors formater_disque() ; Retourner FinSi  
  Lire la structure Disque depuis le fichier
```

Partie III – Implémentation (nouvelle partie)

Introduction

Cette troisième partie est entièrement nouvelle. Elle décrit la mise en œuvre concrète du programme, présente le résultat fonctionnel via des captures d'exécution, et détaille les tests de validation réalisés.

8 Organisation finale du code source

Le code source est découpé en plusieurs fichiers afin de respecter la modularité demandée.

Fichier	Rôle
<code>sgf.h</code>	Toutes les structures (Superbloc, Inode, Bloc, EntreeRep, Disque) et les constantes.
<code>disk.c</code>	Variable globale <code>disque</code> , formatage, recherche d'inodes/blocs libres.
<code>save.c</code>	Sauvegarde et recharge du disque dans le fichier image <code>disk.img</code> .
<code>primitives.c</code>	Toutes les primitives (<code>mycreat</code> , <code>myopen</code> , <code>myread</code> , <code>mywrite</code> , <code>myclose</code> , <code>mymkdir</code> , <code>myrmdir</code> , <code>mylink</code> , <code>myunlink</code>) et les implémentations des commandes shell.
<code>shell.c</code>	Le <code>main</code> et la boucle interactive (REPL) avec le parsing et le dispatcher.
<code>Makefile</code>	Automatisation de la compilation (<code>make</code> , <code>make clean</code> , <code>make mrproper</code>).
<code>test.sh</code>	Script de validation automatique incluant le test de persistance.
<code>.gitignore</code>	Évite de versionner les fichiers générés (<code>*.o</code> , exécutable, <code>disk.img</code>).

9 Détails de l'implémentation

9.1 Gestion du disque simulé

Le disque entier (superbloc + table des inodes + zone de blocs) est stocké dans une unique structure C de taille fixe, déclarée en variable globale dans `disk.c`. Ce choix permet d'écrire ou de relire le disque *d'un seul coup* via `fwrite` et `fread`, sans avoir à gérer d'éléments dynamiques. Le fichier image `disk.img` est créé au premier démarrage par `formater_disque` et systématiquement rechargé par `charger_disque`.

9.2 Primitives fichiers

Les primitives `mycreat`, `myopen`, `myread`, `mywrite` et `myclose` sont implémentées conformément aux algorithmes de la conception. `mywrite` découpe les données en blocs de 1024 octets et alloue de nouveaux blocs au fur et à mesure. `myread` parcourt le tableau d'index de l'inode pour reconstituer le contenu.

9.3 Navigation et primitives répertoires

Une variable globale `inode_courant` a été introduite pour gérer le répertoire courant, à la manière du *current directory* d'UNIX. `mymkdir` ajoute systématiquement les deux entrées spéciales `.` (qui pointe vers le dossier lui-même) et `..` (qui pointe vers son parent). Ces entrées permettent la navigation, notamment le retour au parent via `cd ..`.

`myrmdir` refuse de supprimer un répertoire qui contient plus que les deux entrées `.` et `..`, garantissant ainsi qu'on ne perd jamais le contenu d'un dossier par mégarde.

9.4 Mécanisme des liens

Le compteur de liens `nb_liens` de chaque inode mémorise le nombre de noms qui pointent vers ce fichier. `mylink` crée un nouveau nom et incrémente le compteur ; `myunlink` décrémente le compteur et ne libère physiquement les données que lorsque celui-ci atteint zéro. Ce comportement est strictement équivalent à celui de la commande `rm` sous Linux.

9.5 Interpréteur Shell

Le shell repose sur une boucle de type Read-Eval-Print Loop (REPL). À chaque tour, le programme affiche l'invite `[mysh] $`, lit une ligne via `fgets`, la découpe en mots avec `strtok`, puis appelle la commande correspondante.

Une sécurité a été ajoutée pour remplir les arguments non utilisés avec `NULL`, ce qui évite les erreurs de segmentation lorsque l'utilisateur oublie un argument (par exemple `cd` sans destination).

9.6 Métadonnées et commande `ls -l`

Chaque inode contient ses métadonnées : type (fichier/répertoire), droits, taille, date de modification (timestamp UNIX) et nombre de liens. La commande `ls -l` a été ajoutée pour afficher ces informations sous une forme lisible :

```
[mysh] $ ls -l
d 755      0 octets  25/05/2026 02:37  documents
d 755      0 octets  25/05/2026 02:37  images
- 644      7 octets  25/05/2026 02:37  salut.txt
- 644      7 octets  25/05/2026 02:45  test.txt
d 755      0 octets  25/05/2026 02:45  dossier
```

La conversion du timestamp en texte est effectuée via `localtime` et `strftime`.

10 Démonstration : captures d'exécution

Cette section présente plusieurs sessions réelles capturées dans le terminal pour démontrer le bon fonctionnement du SGF.

10.1 Compilation via le Makefile

```
$ make
gcc -Wall -g -c disk.c
gcc -Wall -g -c save.c
gcc -Wall -g -c primitives.c
gcc -Wall -g -c shell.c
gcc -Wall -g -o sgf disk.o save.o primitives.o shell.o
```

Aucun avertissement (warning) n'est signalé par gcc malgré l'option -Wall, témoignant de la qualité du code.

10.2 Lancement et création de fichiers

```
$ ./sgf
=== Mini-Shell SGF (tapez 'exit' pour quitter) ===
Aucun disque trouve : creation d'un nouveau disque.
Disque formate : 63 inodes et 256 blocs disponibles.
[mysh] $ mkdir documents
Repertoire 'documents' cree (inode 1).
[mysh] $ mkdir images
Repertoire 'images' cree (inode 2).
[mysh] $ echo bonjour > salut.txt
Fichier 'salut.txt' cree (inode 3).
[mysh] $ ls
documents/  images/    salut.txt
[mysh] $ cat salut.txt
bonjour
```

10.3 Navigation dans l'arborescence

```
[mysh] $ cd documents
[mysh] $ echo cours > systeme.txt
Fichier 'systeme.txt' cree (inode 4).
[mysh] $ ls
./  ../  systeme.txt
[mysh] $ cd ..
[mysh] $ ls
documents/  images/    salut.txt
```

10.4 Copie, déplacement et liens

```
[mysh] $ cp salut.txt copie.txt
Fichier 'copie.txt' cree (inode 5).
'salut.txt' copie vers 'copie.txt'.

[mysh] $ mv copie.txt renomme.txt
Fichier 'renomme.txt' cree (inode 6).
'copie.txt' copie vers 'renomme.txt'.
Fichier 'copie.txt' supprime (inode 5 libere).
```

```
'copie.txt' renomme en 'renomme.txt'.  
  
[mysh] $ ln salut.txt alias.txt  
Lien 'alias.txt' cree vers 'salut.txt' (inode 3, 2 liens).
```

10.5 Informations sur le disque (df)

```
[mysh] $ df  
--- Informations du disque ---  
Inodes : 58 libres / 64 total  
Blocs : 252 libres / 256 total  
Espace libre : 258048 octets
```

10.6 Persistance : preuve par l'exemple

Le programme est relancé après un `exit`. Les fichiers créés doivent toujours être présents :

```
[mysh] $ exit  
Au revoir !  
  
$ ./sgf  
=== Mini-Shell SGF (tapez 'exit' pour quitter) ===  
Disque recharge depuis disk.img.  
[mysh] $ ls  
documents/ images/ salut.txt renomme.txt alias.txt  
[mysh] $ cat salut.txt  
bonjour
```

La persistance est ainsi démontrée : après un redémarrage complet du programme, toute l'arborescence et le contenu des fichiers sont préservés.

11 Tests de validation

Conformément au cahier des charges, un script de validation automatique (`test.sh`) a été développé. Il lance le programme, lui injecte une série de commandes via une redirection (*here-document*) et vérifie le comportement du SGF, y compris la persistance entre deux exécutions.

11.1 Catégories de tests couvertes

Catégorie	Description
Création / suppression	Créer des fichiers et répertoires, vérifier les compteurs du superbloc, supprimer, vérifier la libération.
Lecture / écriture	Écrire dans un fichier, le relire et comparer le contenu.
Liens	Créer un lien, supprimer le nom original, vérifier que le fichier reste accessible via le second nom.
Répertoires	Naviguer avec <code>cd</code> et <code>cd ..</code> , vérifier l'isolation entre dossiers.
Refus de suppression	Tenter de supprimer un répertoire non vide : doit être refusé.
Persistance	Créer des fichiers, sortir, relancer, vérifier que tout est restauré à l'identique.
Robustesse	Tester des commandes invalides (<code>cd</code> sans argument, fichier inexistant) : aucune erreur de segmentation.

11.2 Résultat de l'exécution du script

```
=====
TESTS DE VALIDATION DU MINI-SGF
=====
>>> Compilation du projet... Compilation OK.

>>> SESSION 1 : creation de fichiers et dossiers
Repertoire 'documents' cree (inode 1).
Repertoire 'images' cree (inode 2).
Fichier 'salut.txt' cree (inode 3).
documents/  images/  salut.txt
--- Informations du disque ---
Inodes : 60 libres / 64 total
Blocs  : 253 libres / 256 total

>>> SESSION 2 : verification de la PERSISTANCE
Disque recharge depuis disk.img.
documents/  images/  salut.txt
bonjour
=====
FIN DES TESTS
=====
```

La session 2, lancée après un arrêt complet du programme, retrouve l'intégralité de l'arborescence créée en session 1. La persistance est validée.

11.3 Qualité du code

Le code a été compilé avec l'option `-Wall` sans qu'aucun avertissement ne soit signalé. Cela témoigne de la rigueur des bonnes pratiques appliquées : gestion sécurisée des chaînes avec `strncpy`, vérification systématique des valeurs de retour, libération propre des ressources, programmation défensive.

Partie IV – Planning et répartition des tâches

12 Planning du projet

Le projet s'est déroulé sur huit semaines, du 1^{er} avril 2026 au 29 mai 2026. Le planning prévu a été respecté.

Phase	Période	Activités principales
Analyse	1 – 20 avril	Étude du sujet, définition des besoins, conception préliminaire, rédaction du rapport d'analyse.
Conception	21 avril – 2 mai	Conception détaillée des structures et algorithmes, choix d'implémentation, rédaction du rapport de conception.
Implémentation	3 – 23 mai	Codage des modules, intégration progressive, tests unitaires, débogage.
Tests et finalisation	24 – 28 mai	Mise au point des tests de validation, corrections finales, vérification de la persistance, rédaction du rapport final.
Soutenance	29 mai	Présentation orale et démonstration du projet, remise du code source.

13 Répartition des tâches

Le travail a été réparti entre les quatre membres de l'équipe selon leurs spécialités. La répartition décrite ici est la répartition finale, qui reflète les contributions réelles à l'issue du projet.

Membre	Rôle	Contributions principales
Yassine JOUDI	Chef de projet, architecture, persistance	Architecture globale du projet, gestion du dépôt Git/GitHub, conception du superbloc et du formatage, implémentation des fonctions <code>sauvegarder_disque</code> et <code>charger_disque</code> .
Marc ELYSÉE	Développeur SGF et tests	Gestion de la table des inodes, allocation et libération des blocs, gestion des métadonnées (droits, dates), conception et écriture du script de validation <code>test.sh</code> .
Ahmed ISHAGH	Développeur primitives	Implémentation de toutes les primitives fichiers (<code>mycreat</code> , <code>myopen</code> , <code>myclose</code> , <code>myread</code> , <code>mywrite</code>) et répertoires (<code>mymkdir</code> , <code>myrmdir</code> , <code>mylink</code> , <code>myunlink</code>), gestion du compteur de liens.
Ayman MSIMAR	Développeur Shell	Parsing des commandes, dispatcher, implémentation des commandes <code>ls</code> , <code>ls -l</code> , <code>cd</code> , <code>cat</code> , <code>cp</code> , <code>mv</code> , <code>rm</code> , <code>ln</code> , <code>echo</code> , <code>df</code> .

La coordination de l'équipe s'est appuyée sur un dépôt GitHub partagé, permettant le travail simultané sur plusieurs modules et l'intégration progressive des contributions via des *pull requests*.

14 Récapitulatif des évolutions

Voici l'ensemble des évolutions apportées par rapport aux rapports d'analyse et de conception :

Élément	Rapport	Évolution
<code>inode_courant</code>	Conception	Ajout d'une variable globale dédiée pour le répertoire courant (indispensable au bon fonctionnement de <code>cd</code>).
<code>compter_entrees</code>	Conception	Ajout d'une fonction auxiliaire pour vérifier qu'un répertoire est bien vide avant suppression.
<code>ls -l</code>	Analyse	Ajout d'une commande dédiée à l'affichage des métadonnées, conforme à la directive « ajouter des métadonnées sur les inodes ».
<code>echo</code>	Conception	Version simplifiée retenue (un seul mot) pour la livraison ; la version avec guillemets reste un bonus envisageable.
Sécurité <code>args</code>	Implémentation	Initialisation explicite des arguments non utilisés à <code>NULL</code> pour éviter les erreurs de segmentation.

Aucune fonctionnalité prévue n'a été retirée. L'implémentation est conforme au cahier des charges initial, avec quelques ajouts qui renforcent la robustesse et l'ergonomie.

15 Difficultés rencontrées et solutions

Plusieurs difficultés ont été surmontées durant le développement.

- **Gestion des accolades en C** : plusieurs erreurs de compilation (*end of input*) ont été causées par des accolades manquantes. La méthode consistant à inspecter la fin du fichier avec `cat -n fichier.c | tail` s'est révélée efficace pour les localiser rapidement.
- **Erreurs de segmentation** sur des arguments manquants (par exemple `cd` seul) : résolues en initialisant explicitement à `NULL` les éléments inutilisés du tableau `args` après le parsing.
- **Test de non-fuite trompeur** : un premier test de validation indiquait à tort une fuite d'un bloc. Une analyse fine a montré qu'il s'agissait du bloc alloué une fois pour la racine (comportement normal), et non d'une fuite. Le test a été corrigé pour mesurer l'état de référence après la première allocation.
- **Coordination du dépôt Git** : la synchronisation entre les versions des quatre membres a nécessité l'utilisation de branches et de *pull requests* pour intégrer proprement les contributions sans perte de travail.